altairsim — Learning the Symbolic Debugger

an examples/debugger walkthrough

A 46-byte program, its symbols, and a walk through the monitor's debugger — load it, disassemble it by name, single-step it, break on a label, and run it until it prints.

```
cd examples/debugger
altairsim debugger.toml
```

You land at the `altairsim>` prompt with an empty machine: an 8080, 32K of RAM, and an 88-2SIO for the console. Nothing is loaded yet — that is the first thing you will do, and the point of the exercise is that you do it all by hand.

The program is `HELLO.ASM`: it prints `HELLO, WORLD` through the 2SIO and halts. It is small on purpose, so that every label, every `EQU`, and every kind of operand is right there to be looked at.

1 — Load the symbols, then the program

Two separate files, because they are two separate things. `HELLO.PRN` is the assembler's **listing** — it is where the *names* live. `HELLO.HEX` is the assembled **bytes**. Loading the symbols does not put a single byte in memory; loading the hex does not teach the monitor a single name.

```
altairsim> SYMBOLS LOAD HELLO.PRN
12 symbol(s) from HELLO.PRN
altairsim> LOAD HELLO.HEX
loaded 46 bytes from HELLO.HEX (0100-012D)
```

2 — Disassemble it, and read it by name

`DISASM` takes a range, and a symbol is accepted anywhere an address is — so name the range:

```
altairsim> DISASM START-DONE
START:
0100  31 3E 01  LXI SP,STACK
0103  21 1F 01  LXI H,MSG
LOOP:
0106  7E        MOV A,M
0107  B7        ORA A
0108  CA 12 01  JZ DONE
010B  CD 13 01  CALL PUTC
010E  23        INX H
010F  C3 06 01  JMP LOOP
DONE:
0112  76        HLT
```

Three things are happening on top of the plain hex, and each has a rule worth knowing.

A label heads its own line. `START:`, `LOOP:`, `DONE:` — the way an assembler listing prints them, so a jump destination announces itself where it lands.

An operand reads as the name it points at. `JZ DONE`, `CALL PUTC`, `JMP LOOP`, `LXI H,MSG` — every 16-bit address the program references is shown as a symbol.

`LXI SP,STACK` is annotated even though `STACK` is an `EQU`, not a label. This is the same reason `CALL 0005` reads as `CALL BDOS` in CP/M: a program *label* heads a line, but an *operand* is a value the instruction points at, and there an `EQU` that is really an address is exactly what you want to see. (A real label still wins when both share a value.)

Now disassemble the subroutine, and watch what does **not** get named:

```
altairsim> DISASM PUTC 7
PUTC:
0113  F5        PUSH PSW
PWAIT:
0114  DB 10     IN 10
0116  E6 02     ANI 02
0118  CA 14 01  JZ PWAIT
011B  F1        POP PSW
011C  D3 11     OUT 11
011E  C9        RET
```

`IN 10` is **not** shown as `IN TTYS`, even though `TTYS EQU 10H` is loaded (`SHOW SYMBOLS TTYS` proves it). That is deliberate: a port is a **byte**, not an address, and only a 16-bit operand is treated as an address. The count you meant is not the location you didn't — so `ANI 02` stays `02`, not `TXRDY`, and `JZ PWAIT` right below it still reads as the label, because *that* operand is an address.

3 — Single-step, and watch the registers

`EXAMINE` loads the program counter — it is the front-panel switch that jams an address into the PC — so `EXAMINE START` puts you at `0100`. Then `STEP` runs one instruction and shows the machine, with the next instruction disassembled symbolically:

```
altairsim> EXAMINE START
0100  31  1  00110001
altairsim> STEP 3
C0Z0M0E0I0 A=00 B=0000 D=0000 H=0000 S=0000 IE=0 P=0100  LXI SP,STACK
C0Z0M0E0I0 A=00 B=0000 D=0000 H=0000 S=013E IE=0 P=0103  LXI H,MSG
C0Z0M0E0I0 A=00 B=0000 D=0000 H=011F S=013E IE=0 P=0106  MOV A,M
C0Z0M0E0I0 A=48 B=0000 D=0000 H=011F S=013E IE=0 P=0107  ORA A
```

Read down the `P=` column: the stack pointer `S=` fills in after `LXI SP,STACK`, `H=` becomes `011F` (that is `MSG`) after `LXI H,MSG`, and `A=48` after `MOV A,M` — `48` is `'H'`, the first byte of the string. `STEP` with no count does one instruction; `NEXT` is the same but runs a `CALL` to its return instead of descending into it.

4 — Break on a name, and run

A breakpoint takes a symbol, too. Set one on the subroutine and run:

```
altairsim> EXAMINE START
altairsim> BREAK PUTC
breakpoint 1: pc     0113
altairsim> RUN
breakpoint 1 (pc     0113) -- stopped at 0113
6 instructions, 58 T-states.
C0Z0M0E1I0 A=48 B=0000 D=0000 H=011F S=013C IE=0 P=0113  PUSH PSW
```

It stopped the first time the program reached `PUTC`, with `A=48` — the `'H'` it was about to send. Clear it and set one on `DONE` instead, and this time let it finish:

```
altairsim> NOBREAK 1
breakpoint 1 cleared.
altairsim> EXAMINE START
altairsim> BREAK DONE
breakpoint 2: pc     0112
altairsim> RUN
HELLO, WORLD

breakpoint 2 (pc     0112) -- stopped at 0112
C0Z1M0E1I0 A=00 B=0000 D=0000 H=012D S=013E IE=0 P=0112  HLT
```

There it is: the program ran, printed `HELLO, WORLD` through the 2SIO, and stopped exactly at `DONE` — the byte *before* it would have executed the `HLT`. Type `RUN` once more with no breakpoint set and it runs the `HLT` and stands there halted, which is the machine waiting for you.

5 — A few more you now have

Command	Try
`DUMP MSG`	the string, dumped by name — `DUMP` resolves a symbol but does not annotate the bytes, because there is no instruction there to say which are addresses
`SHOW SYMBOLS`	all twelve; `SHOW SYMBOLS T*` filters with a glob
`BREAK PUTC IF A==2C`	a conditional breakpoint — stop at `PUTC` only when it is about to send `2C`, the `,`; the machine prints `HELLO` and stops there
`TRACE ON` / `HISTORY`	log every cycle as it runs, or show the run-up to a stop

`SYMBOLS CLEAR` forgets the names again; disassemble after that (`DISASM 100` — with the names gone you are back to typing the address) and every operand is plain hex once more, which is a good way to see exactly how much the symbols were buying you.

The files

File	What it is
`debugger.toml`	The machine: an 8080, 32K of RAM, and a 2SIO console. No ROM, no disk — nothing you do not need to single-step a program.
`HELLO.ASM`	The source, for reading. Labels, `EQU`s, a loop, and a subroutine.
`HELLO.PRN`	The assembler **listing** — the file `SYMBOLS LOAD` reads. Labels feed the name→line and name→operand annotation; `EQU`s feed operands only.
`HELLO.HEX`	The assembled **bytes**, in Intel HEX — the file `LOAD` reads.

`HELLO.ASM` and `HELLO.PRN` are the same program seen two ways: the listing is the source with the assembler's address and object-code columns added on the left. Edit the source and reassemble, and both the bytes and the names move together.